

Makara Ramoabi

20240045

C++

1. Scenario Analysis

In a role-playing game system, inheritance is used to organize different character types efficiently. A base class called `Character` contains common attributes such as name, health, level, and basic methods like `attack()` and `defend()`. Derived classes such as `Warrior`, `Mage`, and `Archer` inherit from `Character`. Each subclass extends the base functionality by adding unique abilities. For example, `Warrior` may have a `heavyStrike()` method, `Mage` may implement `castSpell()`, and `Archer` may include `shootArrow()`. Although all characters share common properties, their behaviors differ through specialized functions or overridden methods. This structure reduces code duplication and improves maintainability. If the developer updates the health system in the `Character` class, all subclasses automatically benefit. Inheritance therefore promotes reusability, scalability, and logical organization, making it ideal for managing complex game systems with many character types.

2. Concept Research

Multiple inheritance is an object-oriented programming concept where a class inherits from more than one base class. This allows a derived class to combine features and behaviors from multiple parent classes. For example, a `FlyingCar` class could inherit from both `Car` and `Aircraft` classes, gaining properties of both. While multiple inheritance increases flexibility and code reuse, it can also introduce complexity, such as the “diamond problem,” where ambiguity occurs if two parent classes share a common base class. Languages like C++ support multiple inheritance, often using virtual inheritance to resolve ambiguity and manage shared base classes effectively.

3. Tool Practice

While using Visual Studio Code to write a C++ inheritance program, I created a base class called `Animal` with attributes like name and a method `makeSound()`. Then I created a derived class `Dog` that inherited from `Animal` and added its own method `fetch()`. Compiling and running the program showed that the `Dog` object could access both base and derived class methods. This demonstrated how inheritance promotes code reuse and logical structure. I also learned how to use access specifiers like `public` to ensure proper inheritance. The exercise improved my understanding of class relationships and object-oriented programming principles.

4. UML Diagram Explanation

The UML diagram illustrates an inheritance hierarchy with a base class at the top and derived classes below it. The base class, such as Character, contains shared attributes and methods. Below it, subclasses like Warrior, Mage, and Archer are connected using arrows with hollow triangle heads pointing toward the base class, indicating inheritance. Each subclass includes additional unique methods. The diagram visually represents the “is-a” relationship (e.g., Warrior is a Character). This structure makes it easier to understand class relationships, shared functionality, and specialization. UML diagrams help developers plan, communicate, and organize object-oriented designs effectively.